

Lecture 3: Policies, Timing, Populations

第三讲：策略、时序、群体

Computational Methods for Heterogeneous-Agent Macro

异质性主体宏观的计算方法

Jeffrey Sun

May 13, 2026

University of Toronto

Recap / 回顾

Yesterday 昨天

Discretized b , wrote $\mathcal{T}$ as a Julia function,
and iterated V to a fixed point:

$$V = \mathcal{T}V.$$

V is now a vector on the wealth (b) grid.

我们把 b 离散化，把 $\mathcal{T}$ 写成 Julia 函数，
并迭代到不动点：

$$V = \mathcal{T}V.$$

V 现在是财富 (b) 网格上的一个向量。

Today 今天

1. Get the *policy* c^* from V .
2. Define explicit **timing**: each period decomposes into three **stages**.
3. Add Markov process for income and implement with **matrix multiplication**.
4. Simulate one household, then entire **population of heterogeneous agents**.

1. 从 V 中提取策略 c^* 。
2. 把时序讲清楚：一期可拆成三阶段。
3. 沿某一维施加马尔可夫冲击——一次矩阵乘法。
4. 先模拟一户，再模拟整个群体。

Policy Function / 策略函数

Policy Function: c^*

策略函数 c^*

$$V(b) = \max_{c \in [0, b]} u(c) + \beta V(R(b - c) + w) \quad (\text{Bellman Operator})$$

$$c^*(b) = \arg \max_{c \in [0, b]} u(c) + \beta V(R(b - c) + w) \quad (\text{Policy Function})$$

Same right-hand side; the only difference is
 $\max$ vs. $\arg \max$.

Once V is solved, one $\arg \max$ pass reads
off the policy c^* .

右端完全相同，区别仅在 $\max$ 与
 $\arg \max$ 。

只要求出 V ，再做一次 $\arg \max$ 就能读出
策略 c^* 。

Features of c^*

c^* 的形态

$c^*(b)$ is **increasing** (in this and most models): richer households consume more.

The slope $\partial c^* / \partial b$ is the **Marginal Propensity to Consume (MPC)**: high for poorer households, low for rich.

$c^*(b)$ **单调递增** (本模型与大多数模型皆然): 财富越多消费越多。

斜率 $\partial c^* / \partial b$ 就是**边际消费倾向 (MPC)**: 贫者高, 富者低。

Timing / 时序

Income Shocks

收入冲击

We now add **income** as a **state variable**, following a **Markov process** Π .

The **state** is now (b, w) .

On the computer, each V is now a **matrix**. Rows are wealth b , columns are income w .

现在把**收入**加为一个**状态变量**，并令其服从一个 **马尔可夫过程** Π 。

状态变为 (b, w) 。

在计算机里，每个 V 现在是一个**矩阵**。行对应财富 b ，列对应收入 w 。

Stages

阶段

Each **model feature** can be thought of as a **stage**:

1. Income Shock
2. Income
3. Consumption-Saving

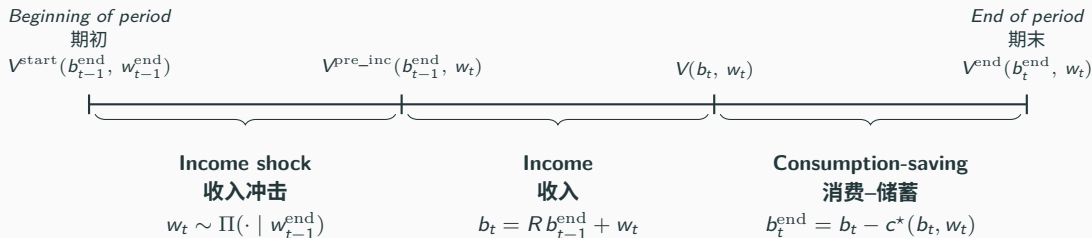
Each stage is a **simple sub-period**.

可以把每个**模型特征**视为一个**阶段**:

1. 收入冲击
2. 收入
3. 消费-储蓄

每个阶段都是一个**简单的子周期**。

Within-Period Timeline 期内时序



Each **stage** has different **state variables** and a different **intermediate value function**.

Because stages are **simple**, we can use them to build **complex** models.

每个阶段有不同的**状态变量**与不同的**中间价值函数**。
正因阶段**简单**，我们才能用它们搭出**复杂模型**。

Stage 1 — Income Shock

第一阶段：收入冲击

$\Pi_{ij} = \Pr(w_t = j \mid w_{t-1}^{\text{end}} = i)$, row-stochastic.

Backward (value functions).

$$V^{\text{start}} = V^{\text{pre_inc}} \Pi^\top.$$

Each $(b^{\text{end}}, w_{t-1}^{\text{end}})$ entry of V^{start} is an expectation over w_t given w_{t-1}^{end} .

后向 (价值函数)。

$$V^{\text{start}} = V^{\text{pre_inc}} \Pi^\top.$$

V^{start} 中每个 $(b^{\text{end}}, w_{t-1}^{\text{end}})$ 项是给定 w_{t-1}^{end} 时对 w_t 取的期望。

Why Matrix Multiplication?

为什么是矩阵乘法？

Hold b^{end} fixed. The values

$$v_j \equiv V^{\text{pre-inc}}(b^{\text{end}}, w_j), \quad j = 1, \dots, N_w$$

form a length- N_w vector.

The Markov expectation is, by definition,

$$V^{\text{start}}(b^{\text{end}}, w_i) = \sum_j \Pi_{ij} v_j.$$

That is exactly the i -th component of Πv —a matrix-vector product.

固定 b^{end} 。值

$$v_j \equiv V^{\text{pre-inc}}(b^{\text{end}}, w_j), \quad j = 1, \dots, N_w$$

构成一个长度 N_w 的向量。

由马尔可夫期望的定义，

$$V^{\text{start}}(b^{\text{end}}, w_i) = \sum_j \Pi_{ij} v_j.$$

这正是 Πv 的第 i 个分量——一次矩阵-向量乘积。

All Wealth Rows at Once 所有财富同时算

Stack the per- b^{end} vectors: $V^{\text{pre_inc}}$ is an $N_b \times N_w$ matrix.

The Markov transition acts *identically* on each row, so the same trick applies in batch:

$$V^{\text{start}} = V^{\text{pre_inc}} \Pi^{\top}.$$

One matrix multiply updates every wealth row.

Markov along one dimension is a single matrix multiplication.

将每个 b^{end} 对应的向量堆叠: $V^{\text{pre_inc}}$ 是 $N_b \times N_w$ 矩阵。

马尔可夫转移对每一行的作用完全相同，于是可以批量处理：

$$V^{\text{start}} = V^{\text{pre_inc}} \Pi^{\top}.$$

一次矩阵乘法即可更新所有财富行。

沿一维做马尔可夫就是一次矩阵乘法。

Stage 2 — Income

第二阶段：收入

$$b_t = R b_{t-1}^{\text{end}} + w_t.$$

Backward (value functions).

$$\begin{aligned} V^{\text{pre-inc}}(b^{\text{end}_{t-1}}, w) &= V(R b^{\text{end}_{t-1}} + w, w) \\ V^{\text{pre-inc}}((b - w)/R, w) &= V(b, w) \end{aligned}$$

后向 (价值函数)。

$$\begin{aligned} V^{\text{pre-inc}}(b^{\text{end}_{t-1}}, w) &= V(R b^{\text{end}_{t-1}} + w, w) \\ V^{\text{pre-inc}}((b - w)/R, w) &= V(b, w) \end{aligned}$$

Stage 3 — Consumption-Saving

第三阶段：消费-储蓄

$$b_t^{\text{end}} = b_t - c^*(b_t, w_t).$$

Backward (value functions). Grid search over b^{end} :

$$V(b, w) = \max_{b^{\text{end}} \in b_{\text{grid}}} u(b - b^{\text{end}}) + V^{\text{end}}(b^{\text{end}}, w).$$

Period boundary: $V^{\text{end}} = \beta V^{\text{start}}$.

后向 (价值函数)。在 b^{end} 上做网格搜索：

$$V(b, w) = \max_{b^{\text{end}} \in b_{\text{grid}}} u(b - b^{\text{end}}) + V^{\text{end}}(b^{\text{end}}, w).$$

期界过渡： $V^{\text{end}} = \beta V^{\text{start}}$ 。

**A Population is a Vector / 群体
即向量**

A Population is a Vector

群体即向量

The same c^* is the rule for every household at every wealth level.

So we can think of a *population* as a distribution λ on the wealth grid:

$$\lambda_i = \text{share of households at } b_i.$$

A population is just a vector —same shape as V , same shape as c^* .

对任何家庭、任何财富水平， c^* 都给出同一条规则。

于是可以把群体视为财富网格上的一个分布 λ ：

$$\lambda_i = \text{处于 } b_i \text{ 的家庭占比.}$$

群体就是一个向量——与 V 、 c^* 同形状。

✓ Is Also Cross-Sectional Welfare

✓ 也是横截面福利

$V(b_i)$ has two readings:

- the value to *one* household if its wealth were b_i ;
- the current welfare of the households who actually have b_i .

Aggregate welfare is then an inner product:

$\bar{V} = \sum_i V(b_i) \lambda_i$. Same for $\bar{C}$, $\bar{K}$, etc.

One V , two uses. Solve once, get aggregates for free.

$V(b_i)$ 有两种读法:

- 设想一个家庭的财富是 b_i , 则其价值;
- 实际财富为 b_i 的那群家庭当下的福利。

总福利就是一次内积: $\bar{V} = \sum_i V(b_i) \lambda_{i0}$
 $\bar{C}$ 、 $\bar{K}$ 等类推。

一个 V , 两种用途。 解一次, 所有总量都送上门。

Simulating a Population / 群体模拟

Forward and Backward, All Three Stages

前向与后向：三阶段汇总

	Backward (value)	Forward (distribution)
Stage 1	$V^{\text{start}} = V^{\text{pre_inc}} \Pi^\top$	$\lambda^{\text{post}} = \lambda^{\text{pre}} \Pi$
Stage 2	$V^{\text{pre_inc}}(b^{\text{end}}, w) = V(R b^{\text{end}} + w, w)$	$(b^{\text{end}}, w) \rightarrow (R b^{\text{end}} + w, w)$
Stage 3	$V(b, w) = \max_{b^{\text{end}}} u(b - b^{\text{end}}) + V^{\text{end}}(b^{\text{end}}, w)$	$(b, w) \rightarrow (b - c^*(b, w), w)$
Boundary	$V^{\text{end}} = \beta V^{\text{start}}$	—

Backward: VFI iterates

$V^{\text{end}} \rightarrow V \rightarrow V^{\text{pre_inc}} \rightarrow V^{\text{start}}$, then loops.

Forward: the population's λ traces the same three stages in time order.

后向: VFI 沿 $V^{\text{end}} \rightarrow V \rightarrow V^{\text{pre_inc}} \rightarrow V^{\text{start}}$ 迭代后再循环。前向: 群体 λ 按时间顺序经历同样的三个阶段。

One Markov Step Along the Income Dimension

沿收入维做一步马尔可夫

```
# λ is an N_b × N_w matrix: rows index wealth, columns index income state
# applying Π along the income dimension is a single matrix multiplication:
λ_new = λ * Π
```

The mass at (b_i, w_j) flows out to $(b_i, w_{j'})$ with weight $\Pi_{jj'}$.

Wealth doesn't move in this step —that's Stages 2 and 3.

Markov along one dimension is just a matrix multiplication. We'll use this trick repeatedly in later lectures.

(b_i, w_j) 处的质量按权重 $\Pi_{jj'}$ 流向 $(b_i, w_{j'})$ 。财富在这一步不动——那是第二、第三阶段的事。

沿一维做马尔可夫就是一次矩阵乘法。这个套路后面会反复用到。

simulate_population / 群体模拟

```
function simulate_population(c_ind, params, λ_0; T = 200)
    (; R, w, w_vals, b_grid, Π) = params

    # destination wealth index for each (i_b, i_w_next) pair
    dest = snap_idx.(Ref(b_grid), b_grid[c_ind] .- w .+ w_vals')

    λ_path, λ = [copy(λ_0)], copy(λ_0)
    for t in 1:T
        λ_mid = λ * Π                                # Stage 1: Markov on income
        λ      = scatter_into(λ_mid, dest)             # Stages 2 + 3: index lookup
        push!(λ_path, copy(λ))
    end
    return λ_path
end
```

Stage 1 is one matrix multiply. Stages 2–3 send every (i_b, i_w) cell to a single destination (i'_b, i_w) given by the precomputed index matrix `dest` — no lotteries.

第一阶段是一次矩阵乘法。第二、第三阶段按预先算好的下标矩阵 `dest`，把每个 (i_b, i_w) 单元送到唯一的目标 (i'_b, i_w) ——不用抽奖。

The Long-Run Distribution

长期分布

Apply the forward step over and over. After enough periods the distribution λ stops changing.

The limit distribution λ^* describes the long-run population. Once we have it, aggregates are just inner products with b , V , c^* , etc.

反复施加一步前向算子。期数足够多后， λ 不再变化。

极限分布 λ^* 描述了群体的长期形态。有了它，总量就是与 b 、 V 、 c^* 等做内积。

Recap / 回顾

Recap

回顾

- c^* from V : same operator, $\arg \max$ in place of $\max$.
- Each period decomposes into three stages: shock, income, choice.
- Markov along one dimension is a matrix multiplication.
- Once we have c^* we can roll a single household forward; once we have V we have cross-sectional welfare for free.
- 从 V 提取 c^* : 算子不变, 把 $\max$ 换成 $\arg \max$ 。
- 一期分三阶段: 冲击、收入、决策。
- 沿一维做马尔可夫就是矩阵乘法。
- 有了 c^* 就能推进个体, 有了 V 就免费得到横截面福利。

Tomorrow: Equilibrium

明天：均衡

So far prices (R, w) have been exogenous.
Tomorrow we let prices respond to λ^* .

That closes the macro loop:

$$\lambda^* \iff (R, w).$$

The result is the Aiyagari model.

到目前为止价格 (R, w) 是外生的。明天让
价格响应 λ^* 。

宏观回路这就闭合：

$$\lambda^* \iff (R, w).$$

这就是 Aiyagari 模型。